Mechanized Liveness Verification of an Asynchronous Runtime: The `lion-liveness` Formal Model in Verus

Ti Zhou

July 26, 2026

Abstract

We present a mechanized liveness proof for an asynchronous runtime. We decompose the runtime into a *runtime Core* (the Executor and the Reactor), an *Injection Queue* (the external queue into which tasks are submitted and from which the Executor draws them), and a set of *Tasks* that are managed by the Core and interact through synchronization primitives; the observable state of the runtime under this decomposition is modeled as the composition of the modules' event logs. Under three classes of premises—(i) *environment assumptions* (for example, that the clock advances monotonically; the remaining assumptions are introduced in later sections), (ii) *user task-behavior assumptions* (for example, that a task returns ready within a bounded number of polls), and (iii) *Core invariants* (the module invariants of the Reactor and the Executor, whose validity is discharged by the separately verified implementation crates `lion-reactor` and `lion-executor`)—we prove that every task injected into the Injection Queue is driven to completion, that is, is polled until it returns `Ready`.

Contents

1 Model: Event Logs and Async Contracts

Log model. Every module is modeled by its *event log*: a finite sequence $\ell \in L$ recording the events the module has emitted so far. A module is specified by a pair

$$\mathsf{ModuleSpec}\langle L\rangle = \big(wf : L \to \mathbb{B},\ \ progress : L \times L \to \mathbb{B}\big),$$

where wf is a well-formedness invariant on logs and $progress(\ell, \ell')$ is the one-step transition relation (typically ℓ' extends ℓ by a single event). We write $progress^{\,n}(\ell, \ell')$ when a valid trace $\ell = \ell_0, \ell_1, \ldots, \ell_n = \ell'$ with $progress(\ell_i, \ell_{i+1})$ for all i connects the two logs, and call wf *inductive* when $wf(\ell) \wedge progress(\ell, \ell') \Rightarrow wf(\ell')$.

Async contracts. A liveness obligation is expressed as an *async contract* over a subject type T (a task id, a resource id, ...):

$$\mathsf{AsyncContract}\langle L, T\rangle = \big(acc,\ ful,\ asm\big), \qquad acc,\ ful,\ asm : L \times T \to \mathbb{B}.$$

Read operationally: once the trigger $acc(\ell, t)$ (*acceptance*) has occurred and the assumption $asm(\ell, t)$ holds, the module must eventually reach the goal $ful(\ell, t)$ (*fulfillment*).

Bounded liveness under an environment filter. The goal must be reached within a *bounded* number of steps along every run whose *every* state satisfies the assumption. Writing $progress^{\,n}_{asm}(\ell, \ell')$ for an n-step trace $\ell = \ell_0, \ldots, \ell_n = \ell'$ all of whose states satisfy $asm(\ell_i, t)$, a module *satisfies* its contract when it preserves its invariant and, from every accepted state at which the assumption holds, some horizon n makes the goal unavoidable:

$$\begin{aligned}\mathsf{BoundedLiveness}(ms, ac) \ \equiv\ & (wf \text{ inductive}) \ \wedge\ \forall t, \ell.\ \big(wf(\ell) \wedge acc(\ell, t) \wedge asm(\ell, t)\big) \\ & \Rightarrow\ \exists n.\ \forall \ell'.\ progress^{\,n}_{asm}(\ell, \ell') \Rightarrow ful(\ell', t).\end{aligned}$$

This $\Box\, asm \Rightarrow \Diamond\, ful$ schema is the single shape in which every module's liveness—and the composed end-to-end guarantee—is stated and proved. Concretely, it is the following Verus specification, where `env` is the filter asm and `env_progress_n` is the env-filtered reachability $progress^{\,n}_{asm}$:

Listing 1: The Verus definition of BoundedLiveness (env-filtered form). Here `env` abbreviates the *environmental assumption*; `progress_preserves_wf` requires `well_formed` to be preserved by *every* progress step; SMT trigger annotations are elided.

```
pub open spec fn env_response_within_trace<L, T>(
  l: L, t: T, ms: ModuleSpec<L>, ac: AsyncContract<L, T>,
  env: spec_fn(L, T) -> bool, n: nat,
) -> bool {
  forall |l_prime: L|
    env_progress_n(ms.progress, l, l_prime, n, env, t)
      ==> (ac.fulfillment)(l_prime, t)
}

pub open spec fn bounded_liveness_env_without_arrival<L, T>(
  ms: ModuleSpec<L>, ac: AsyncContract<L, T>,
  env: spec_fn(L, T) -> bool,
```

```
) -> bool {
  progress_preserves_wf(ms) &&
  forall |t: T, l: L|
    (ms.well_formed)(l) &&
    (ac.acceptance)(l, t) &&
    env(l, t) ==>
    exists |n: nat| env_response_within_trace(l, t, ms, ac, env, n)
}
```

2 State

The entire observable state of the runtime is a single record of per-module event logs. Each log is a sequence of that module's events—an instance of the log model L of Section 1—so a state records *what each module has done so far*, and progress appends events to these logs.

Listing 2: The composed runtime state. In the source, the sequence types appear as the aliases `ExecutorLog` / `ReactorLog` / `TaskLog`.

```
pub struct ComposedState {
  pub executor_log:       Seq<ExecutorEvent>,          // Executor
  pub reactor_log:        Seq<ReactorEvent>,           // Reactor
  pub task_logs:          Map<TaskId, Seq<UtilityEvent>>,  // managed Tasks
  pub injection_schedule: Seq<Task>,                   // Injection Queue (ghost)
}
```

The four fields realize the decomposition directly:

- `executor_log` and `reactor_log` are the two logs of the runtime *Core*—the Executor's scheduling events (spawn/pop/poll/drain) and the Reactor's timer and I/O events (register/set-waker/wake).
- `task_logs` maps each live task id to that task's own log of interactions with the synchronization primitives—the *Tasks* managed by the Core; its key set is exactly the currently-managed tasks.
- `injection_schedule` is a *ghost* model of the external *Injection Queue*: the committed order in which submitted tasks are delivered. It is fixed across progress, and the Executor's pops consume it in order.

Identities cross module boundaries; objects do not. Each object is owned by one module and stays there—task futures in the Executor (keyed by `TaskId`), I/O sources and timers in the Reactor (keyed by `ResourceId`)—and modules name one another's objects only by id. Hence the modeled state is logs of *events over ids* (Listing 2): the objects never enter the composed state, only their identities do.

3 Progress

A *progress step* $s \to s'$ is the composed runtime's one-step transition—the *progress* relation of Section 1 instantiated for `ComposedState` (in code, `composed_progress`). It advances the executor, reactor, and per-task logs at once, and holds exactly when four conditions are met.

1. **Logs only grow.** Each log in s'—the executor's, the reactor's, and every task's—has its counterpart in s as a prefix, and the injection schedule is unchanged: a step *appends* events, it never rewrites history.

2. **Each module takes its own step.** The three modules advance by their own, separately verified semantics: the executor sublog by one legal executor step, the reactor sublog by one legal reactor step, and each task's log—which records that task's use of the synchronization primitives—grows only when the executor polls the task (condition 3) while always preserving that task's own invariant (correct ownership of the timers and IO it registered, and its wakeup structure).

3. **The modules' logs stay aligned.** The executor and reactor advance independently, yet they narrate the *same* history from two sides; without a constraint their logs could drift into contradictory stories, so a step also requires:

   - *Action mediation.* Every reactor operation a task performs—registering or deregistering a timer/IO resource, setting a waker—matches exactly one event on the reactor side, in the same order (a one-to-one correspondence), and a resource is only deregistered by the task that registered it.
   - *Observation consistency.* Whenever the executor polls a task, that poll shows up in the task's own log and the two agree—in particular, a poll returning `Pending` leaves the task's log ending in `Pending`. The executor's view of a task is exactly what the task recorded about itself.

4. **Wakeups are delivered.** Progress is only useful if a wake actually reaches the runnable queue. This is the one place a step *assumes* cross-module behavior rather than deriving it: when the reactor issues a wake for a task, that task lands in the matching pipeline (e.g. the `ReactorWake` queue), which a later executor drain then observes. So a step must carry every fired-but-unconsumed wake through to a poll:

   - *Drain delivery.* A fired wake—from a task that yielded (deferred), from an expired timer or ready IO (a reactor `WakeTask`), or from a waker another task invoked (cross-task)—is picked up by this step's corresponding queue drain. So "fired $\Rightarrow$ queued this step" is built into progress, not left as a wish.
   - *Injection delivery.* The executor's task pops follow the committed injection schedule, so a submitted task is really handed to the runtime.
   - *Park sync.* The executor and reactor agree on when the runtime blocks (equal park counts), so no wake is lost across a block.

   We model each pipeline as a trusted FIFO boundary rather than a state object shared by two modules: these queues are internal to the runtime and never disturbed from outside, so capturing only their FIFO behavior is faithful while sidestepping the semantics of cross-module shared state.

This step relation is the `progress` field of the composed module spec, so the schema's $progress^{n}_{asm}$ is its n-fold composition.

4 Assumptions

The theorem is conditional: it guarantees completion only when the world around the runtime, the user's tasks, and the Core modules all behave. These premises fall into three kinds.

- **Environment assumptions.** Facts about what happens *outside* the runtime, which no internal logic can force. The clock keeps advancing (timestamps strictly increase—an idealization: the implementation clock is millisecond-granular, so runs where two clock reads observe the same millisecond fall outside the assumed environment; see `TCB_and_limitations.md`); every wake source a task registers eventually fires—a timer's deadline is reached, a registered IO becomes ready within a bounded number of polls, a waker handed to another task is eventually invoked; and submitted tasks are delivered to the executor in a fixed order. In reality any of these can fail—an awaited IO that never arrives—which is exactly why they must be assumed rather than proved.
- **User task-behavior assumptions.** Constraints on how the user's own tasks behave. The central one is that a task returns `Ready` within a bounded number of polls—a task that loops forever legitimately never completes, so completion cannot be promised without it. We also assume a task does not abandon a resource it is waiting on (it keeps its timer or IO registered until woken), and that task identities are unique.
- **Core invariants.** Properties the runtime's own modules maintain: correct FIFO scheduling and queue draining in the Executor, correct timer/IO management and timely wakeup in the Reactor, and correct per-task resource ownership. The liveness proof *uses* these as given; their validity is discharged by the separately verified implementation crates `lion-executor` and `lion-reactor`.

5 The End-to-End Liveness Theorem

Everything above assembles into a single statement—an instance of the BoundedLiveness schema of Section 1 with the subject T ranging over task ids. The four ingredients are exactly the pieces built up so far: *acceptance* is "*tid* has been submitted to the injection queue (and not yet dequeued)"—precisely: *tid* heads the committed injection schedule, so the very next tick delivers it (the depth-k variant below lifts the head-of-schedule restriction); *fulfillment* is "*tid* has been polled to `Ready`—it has completed"; the *assumption* is the environment of Section 4; and *well-formedness* is the Core invariants and cross-module consistency of Section 3. In words:

> For every well-formed state s and task *tid* that has been submitted to the injection queue, and for every completion bound $cap \geq 1$ (the task returns `Ready` within *cap* polls), there is a step budget n such that *every* run of n progress steps from s along which the environment keeps cooperating ends in a state where *tid* has completed.

Formally, writing $progress^{n}_{\mathsf{env}_{cap}}$ for the n-step reachability all of whose states satisfy the trace-level environment (Section 1; env_{cap} strengthens the single-state env with the *cap*-poll completion bound and the io-readiness and cross-task-wake arrival clauses):

$$\forall\, s, tid, cap \geq 1.\ \ \mathsf{wf}(s) \wedge \mathsf{submitted}(s, tid) \wedge \mathsf{env}(s, tid) \\ \Rightarrow\ \exists\, n.\ \forall\, s'.\ progress^{n}_{\mathsf{env}_{cap}}(s, s') \Rightarrow \mathsf{ready}(s', tid).$$

A few points fix the strength of the guarantee:

- *Bounded and unavoidable.* Within a single fixed horizon n, *every* cooperating continuation—not merely some—reaches `Ready`; the task cannot starve while the environment behaves. The horizon is concrete: a mechanized corollary (`ete_reaches_goal_explicit_bound`) exhibits $n = \mathit{chunk} + \mathit{cap} \cdot \mathit{chunk}$ with $\mathit{chunk} = K + B + C + \mathit{cap} + 2$, where K bounds a timer's deadline in scheduler rounds, B bounds io readiness in polls, and C bounds the runnable-queue length—the three environment constants of Section 4. The bound mentions nothing about the goal, so it cannot be circular.

- *Every completion bound.* The *cap* is quantified universally, so the theorem covers every task that finishes in *some* finite number of polls; a task that never returns `Ready` is, correctly, not promised completion.

- *Conditional on a cooperating run.* This is a $\Box\,\mathsf{env} \Rightarrow \Diamond\,\mathsf{completed}$ statement: the guarantee holds precisely along runs whose every state satisfies the environment assumption.

- *Well-formedness is inductive.* `progress_preserves_wf` makes the Core invariants hold at every reached state, so the precondition on s propagates along the whole run for free.

- *Arbitrary queue depth (mechanized generalization).* The submission trigger is also generalized from head-of-queue to an arbitrary position: a proven variant (`end_to_end_liveness_from_depth`) covers a task at any depth k of the committed injection schedule with the enlarged budget $n = |\mathit{schedule}| + \mathit{chunk} + \mathit{cap} \cdot \mathit{chunk}$, the delivery leg being *derived*—each tick performs a pop and the schedule model converts $|\mathit{schedule}|$ pops into full delivery—with no new assumption, and its domain is inhabited by a depth-1 witness run that completes both scheduled tasks in order.

6 Proof Sketch (Every Step Mechanically Verified in Verus)

Three colours mark where each step of the argument gets its force: assumptions (what we take the outside world and the user's task to do), invariants (the honest bookkeeping that the reactor, the executor, and each task keep about themselves), and contracts (the promises the reactor and the executor each make about waking and running tasks). The three are layered: each contract inherits its precondition from the assumptions and leans on the properties the invariants supply. And those invariants are not paper idealizations—they are exactly the properties checked against the runtime's own executable code, in the `lion-reactor` and `lion-executor` crates. The argument then reads as one story.

Picture a task that has just been dropped into the injection queue. The outside world has committed to actually hand that task over to the runtime, and the executor's promise is that it keeps pulling tasks off the queue and running them, so our task is eventually pulled out and run for the first time.

Running the task has just two outcomes. Either it says "done", and we are finished—or it says "not yet". A well-behaved task never pauses silently; whenever it pauses it has arranged a way to be woken again, and there are only a few such ways—each with its own route back onto the executor's waiting list:

- *It set a timer, or it is waiting for data to arrive.* The timer eventually reaches its time, or the awaited data eventually becomes ready; then the reactor promptly produces a wake for our task, aimed correctly because the reactor keeps track of whose wake is whose, and that wake lands our task back on the waiting list.

- *It is waiting for another task to signal it*—say, a message on a channel. That other task eventually sends the signal, and doing so drops our task straight onto the waiting list; this route never goes through the reactor at all. We wrap this cross-task wakeup as its own promise—the same async-contract shape as the reactor's and executor's—but stated at the level of the synchronization primitives, so that each one (a channel, a lock, and so on) can be verified to keep it on its own.

- *It simply stepped aside to let others run*—a voluntary yield. There is nothing to wait for: the task is parked for a moment, and the executor puts it back onto the waiting list on the very next round.

One subtlety hides inside "it has arranged a way to be woken." A single pause may arrange *several* ways at once—a timer *and* an awaited read, say. These are not independent guarantees that all come true: the first one to fire wakes the task, and the very act of waking can retire the others (the future is polled again, drops the read it no longer needs, and that IO registration goes stale). So we cannot claim "*every* source fires"—that is simply false. What we prove is the honest, weaker statement: whenever the task is paused, at least one of its arranged sources is still live—and the proof does not care which. It *selects a single still-live witness*—the earliest source that will actually deliver a wake—and follows only that one route back to the waiting list, ignoring whatever siblings may have gone stale. Concretely the "still-live" bookkeeping bakes in deregistration, and it does so differently for the two kinds of source, matching how each behaves at runtime. A timer must have been *re-armed in this very poll*—Lion re-registers timer interest on every poll—so any timer left over from an earlier poll simply fails to count, no history scan needed. An IO registration may persist across polls, so for it we scan the whole history: it counts only if it was registered and has *not* been cancelled anywhere since. Either way a stale timer or a cancelled read is not even a candidate. And if a future dropped *all* its sources yet still paused, that would break the very invariant that a paused task always has a live source—so such a silently-stuck trace is not well-formed and never enters the theorem at all. This is why the whole argument is phrased as "there *exists* a source that wakes us," never "*all* sources wake us."
In every one of these cases our task ends up back on the waiting list, and the executor promises that anything on that list gets run again, and soon—and because the list is kept in fair order, our task really does reach the front instead of being skipped forever.

So the task gets run a second time—and we are back at the same fork: done, or pause and wait for the next wake. Here is the one thing that makes the whole loop stop: the user's task only ever needs to be run a limited number of times before it finally says "done". Each trip around the loop delivers one more run.

Putting it together: each wait-then-wake-then-run trip takes only a bounded amount of time, and only a limited number of trips are ever needed, so within a fixed budget of steps the task is run often enough to finish. That is exactly the top-level guarantee—a task dropped into the injection queue is driven all the way to completion.

7 The Async Contracts

The proof chains a small catalogue of module contracts, one per wake-or-run path. Each has the same shape—a trigger it accepts, and a goal it then guarantees within a bounded number of progress steps under that contract's assumption. In plain language:

Layer	Contract	Guarantee (within a bounded number of steps)
Reactor	`bounded_timer_wakeup`	Once a timer is registered, the reactor emits a wake for it.
Reactor	`bounded_io_wakeup`	Once a waker is set for an IO resource, the reactor emits a wake carrying that waker when the resource becomes ready.
Executor	`bounded_injection_poll`	Once a task is drawn from the injection queue, the executor polls it.
Executor	`bounded_drain_poll`	Once a task appears in a drain of a runnable queue, the executor polls it; instantiated per queue below.
	`bounded_reactor_wake_poll`	... for the ReactorWake queue (timer / IO wake-ups).
	`bounded_task_wake_poll`	... for the TaskWake queue (cross-task wake-ups).
	`bounded_deferred_poll`	... for the Deferred queue (voluntary yields).
Utility	`passwaker_to_wakewaker_contract`	Once a task hands its waker to a synchronization primitive (a channel, a lock, ...), that primitive invokes the waker.

Table 1: The async contracts of the three layers. The end-to-end theorem chains them into the single guarantee walked through in Section 6.

8 The Invariants

These are the invariants of the proof sketch—the honest bookkeeping each module keeps about itself, and the very properties checked against the executable code of `lion-executor` and `lion-reactor`. Each is a safety property (it holds at every state); together with the cross-module alignment of Section 3 they form the well-formedness precondition of the theorem.

Executor (`executor_inv`). Every tick has a fixed shape: it parks once (`tick_has_park`) and services all of its intake queues—popping the injection queue (`tick_has_pop_injection`) and draining the deferred, task-wake, and reactor-wake queues (`tick_has_drain_deferred`, `tick_has_drain_task_wake`, `park_drain_reactor_wake`). Scheduling is fair and sound: runnable tasks are taken first-in-first-out (`fifo_task_selection`), only valid tasks are ever polled (`valid_task_polling`) and only from inside a tick (`poll_within_tick`), and any task that is runnable is actually polled within the tick (`tick_polls_if_runnable`).

Reactor (`reactor_inv`). The timer and IO bookkeeping stays consistent: each resource id names at most one live timer or IO (`timer_reg_uniqueness`, `io_reg_uniqueness`), a timer and an IO never share an id (`timer_io_disjoint`), every wake traces back to a real registration (`wake_has_registration`), a waker is only ever set on a still-active IO (`set_waker_active_io`), and every waker is valid (`timer_waker_validity`, `io_waker_validity`). Each park cycle is well-formed—it stamps the current time and polls for IO exactly once (`park_has_timestamp`, `park_poll_once`), and a registered timer's deadline lies in the future (`timer_deadline_future`). And, crucially for liveness, wakes are delivered on time: an expired timer (`wake_on_expired`), or an IO that becomes ready in the awaited direction (`wake_on_io_ready_readable` / `wake_on_io_ready_writable`), produces a wake. A further group keeps the IO registration cycle honest—an IO register or deregister syscall is issued only inside the correspond-

ing inbound API cycle (`register_io_in_cycle`, `deregister_io_in_cycle`), the result that cycle returns matches the syscall it actually performed (`inbound_register_io_result`, `inbound_deregister_io_result`), and a readiness report only arises inside a park cycle (`io_ready_in_park`). In all, the reactor carries 19 such properties, split between `reactor_inv` and the extended `reactor_ext_inv`.

Task (`utilities_inv`). Each task keeps two things straight about itself. Whenever one of its polls returns `Pending`, it has already registered an active way to be woken (`wakeup_guarantee`)—the property the proof sketch leans on the moment a task pauses. And it owns exactly the timers and IO it registered, so only it may retire them (`resource_ownership`). A note on scope: user task code is unboundedly varied and cannot itself be modeled, so these properties are not checked against arbitrary user futures. They serve instead as a *specification*—a criterion that the building blocks a task is composed from must meet—and are discharged at the level of the individual utilities and primitives (a `TcpStream`, a `Signal`, a channel, ...) rather than the user's program as a whole.

9 Termination: the Progress Measure

Bounded liveness is at bottom a termination argument, and every termination argument needs a *ranking function*: a quantity that provably strictly decreases and is bounded below, so the system cannot circle forever. Where does ours come from?

Assumption + invariant = provable progress. The measure is induced by two things acting together: the environment assumptions—time is monotonic, a pending timer will eventually expire, the OS notifies the runtime of an incoming event (through `epoll`) within a symbolic bound, a remote TCP peer responds within a symbolic bound—and the runtime's own internal progress invariants. Neither alone suffices: an assumption on its own only says the system is *allowed* to make progress; it takes an invariant to make that progress *happen* on a concrete step. For instance, clock monotonicity (an assumption) says time never runs backwards but says nothing about how fast. Pair it with the invariant "every park cycle reads the clock at least once, through a syscall" (`park_has_timestamp`), and the two together drive the clock strictly forward on *every* cycle. That per-cycle decrease is exactly what turns "a pending timer will eventually expire" into "the timer's deadline is reached within a bounded number of cycles." It is the difference between the system being *allowed* to make progress and *provably* making it.

One outer induction. Stitched together, the whole proof rests on a single induction. Within one poll the chain of module contracts (Table 1) is *acyclic*—a submitted or woken task is drained and then polled, and a poll that returns `Pending` registers a fresh wake source—so there is no circular dependence to untangle inside a poll. The only loop is the outer back-edge: a task returns `Pending`, becomes runnable again, and is polled once more.

The measure on the back-edge. The ranking function for that loop is the maximum number of polls the task may still need before it completes—the bound `cap` of the theorem, supplied by the user task-behavior assumption. Every trip around the back-edge consumes exactly one poll, so this remaining-poll count strictly decreases, and it is bounded below by zero. Each trip is itself bounded in steps—by the symbolic inner bounds above (a timer's deadline in cycles, IO readiness in polls, the queue length)—so a bounded number of bounded trips yields the theorem's finite horizon n. When the count reaches zero the task has returned `Ready`: it is done.

10 The Theorem Is Not Vacuous—and We Machine-Checked It

A universally quantified implication hides a trap. "For every state satisfying P, the goal follows" is *vacuously* true—and says nothing at all—if no state ever satisfies P. A liveness theorem whose assumptions were secretly contradictory, or that quietly covered only tasks that were already finished, would be exactly such an empty statement. We rule this out on purpose. There are two ways a proof like ours could collapse into emptiness.

An assumption no state can satisfy. If a single assumption is unsatisfiable, the whole precondition is unsatisfiable and the theorem rides on a false premise. The subtle offender is an assumption phrased over *all possible futures*—"no matter how the run continues, X holds"—because an adversarial continuation can usually break X, so no state satisfies it. A semantic audit went through every assumption and replaced each such forall-over-futures form (three families of them) with a *per-state* form that a concrete execution can actually meet. No assumption is individually constant-false.

A precondition met only by trivial cases. Even with satisfiable assumptions, the theorem would still be hollow if the only states meeting its precondition were tasks that had *already* completed—they reach the goal in zero steps, for free, so a theorem covering nothing else would be true but content-free. The interesting case is the *not-ready* branch: a task that genuinely still has to wait.

A concrete witness that inhabits the interesting case. We settle both at once by exhibiting an actual, machine-checked execution (`b_domain_inhabited`). It starts from a real state where the task has been submitted but not yet run, and is *not yet ready*—so the not-ready branch of the domain is provably non-empty—while every part of the precondition (well-formedness, submission, and the environment) holds. From there the run is concrete: the task is polled once and returns `Pending`, a timer it set fires and wakes it, it is polled a second time, and it returns `Ready`. This single trace witnesses that the full precondition is satisfiable, that the not-ready case is non-empty, and that the goal is actually reachable.

Together these mean the guarantee has real content: it constrains genuine, waiting executions rather than an empty set of states. The theorem holds because the runtime drives waiting tasks to completion—not because its hypotheses can never be met.

What the witnesses do not close. Two boundaries of this anti-vacuity argument are worth stating plainly. First, the witness executions assume mild lower bounds on the environment's symbolic constants (a queue capacity of at least one, a timer horizon of at least a few cycles); these constants are uninterpreted parameters of the model, so the witnesses establish inhabitation *for every environment whose constants clear those bounds*, not unconditionally. Second, the theorem's conclusion quantifies over cooperating n-step runs ($\Box\,\mathsf{env}$); it does not itself assert that a cooperating run *exists* from every qualifying state. From the witnesses' constructed states this existence *is* proven at every step budget: extending the timer-wake witness with idle scheduler ticks (each an empty tick–park round that preserves the environment) yields a cooperating run reaching `Ready` in exactly n steps for every $n \geq 2$, so the domain of the theorem's conclusion is inhabited at any budget it is instantiated with—including the proof's own n^*. What remains out of scope, deliberately, is such a construction from *arbitrary* qualifying states rather than from the witnesses.

11 Locating Each Claim in the Code

Every claim in this document is a named Verus item; Table 2 maps them (paths relative to `lion-liveness/src/`; the module contracts of Table 1 live in `lion-reactor-spec/src/contracts/`, `executor/contracts/`, and—for the utility contract—`lion-utility/lion-utility-spec/src/generic/contract.rs`).

Claim	Verus item	File
End-to-end theorem (§5)	`end_to_end_liveness_env_N_trace` (proven by `..._holds`)	`composed/proof/assumption_satisfiable.rs`
Arbitrary-depth variant	`end_to_end_liveness_from_depth_trace`	`composed/proof/depth_generalization.rs`
Explicit step bound ($chunk + cap \cdot chunk$)	`ete_reaches_goal_explicit_bound`	`composed/proof/assumption_satisfiable.rs`
Well-formedness preservation	`composed_progress_preserves_wf_lemma`	`composed/proof/end_to_end.rs`
Assumptions jointly satisfiable	`top_theorem_precondition_satisfiable`, `env_N_satisfiable`	`composed/proof/assumption_satisfiable.rs`
Non-vacuity: timer path	`b_domain_inhabited` (all budgets: `..._at_any_n`)	`composed/proof/inhabitation_goal_wake.rs`, `inhabitation_budget.rs`
Non-vacuity: io path	`bio_domain_inhabited`	`composed/proof/inhabitation_goal_io.rs`
Non-vacuity: deferred path	`d_domain_inhabited`	`composed/proof/inhabitation_goal_defer.rs`
Non-vacuity: cross-task path	`t_domain_inhabited`	`composed/proof/inhabitation_goal_taskwake.rs`
Non-vacuity: depth variant	`depth_domain_inhabited`	`composed/proof/inhabitation_depth.rs`
Cross-task arrival clause non-vacuous	`taskwake_arrival_within_nonvacuous_witness`	`composed/proof/inhabitation_goal_taskwake.rs`
Drain occurrence derived	`single_progress_has_drain_*`	`executor/proof/bounded_drain_poll.rs`

Table 2: Paper claim → Verus proof item. `./ci.sh` re-checks all of them.

12 Scope of Trust

Two things are trusted rather than machine-checked, and we state them explicitly. First, the correspondence between the L2S model and the runtime's executable code is *informal*: the invariants are verified against the actual `lion-reactor` and `lion-executor` sources, but there is no mechanized refinement proof that the event-log model captures every behavior of the compiled runtime. Second, the proofs rest on a trusted computing base of `external_body` items—OS and syscall boundaries, waker vtables, and low-level slot operations—whose postconditions are trusted, not verified, plus the Verus toolchain and its SMT solver. The complete, per-item inventory is published as `TCB_and_limitations.md` at the repository root. Known modeling limitations (e.g. the resource-id allocator currently forgoes reuse, trading unbounded slab growth for a simpler no-reuse model) are documented there as well.